🏛 Banking AI Agent

Contextual Engineering in Practice

LangChain · LangGraph · OpenAI · RAG · Multi-Agent

> *Demonstrating all four Contextual Engineering strategies through a production-style banking assistant*

The Core Problem with LLM Agents

LLMs have a **fixed context window** — they can only "see" what you put in front of them.

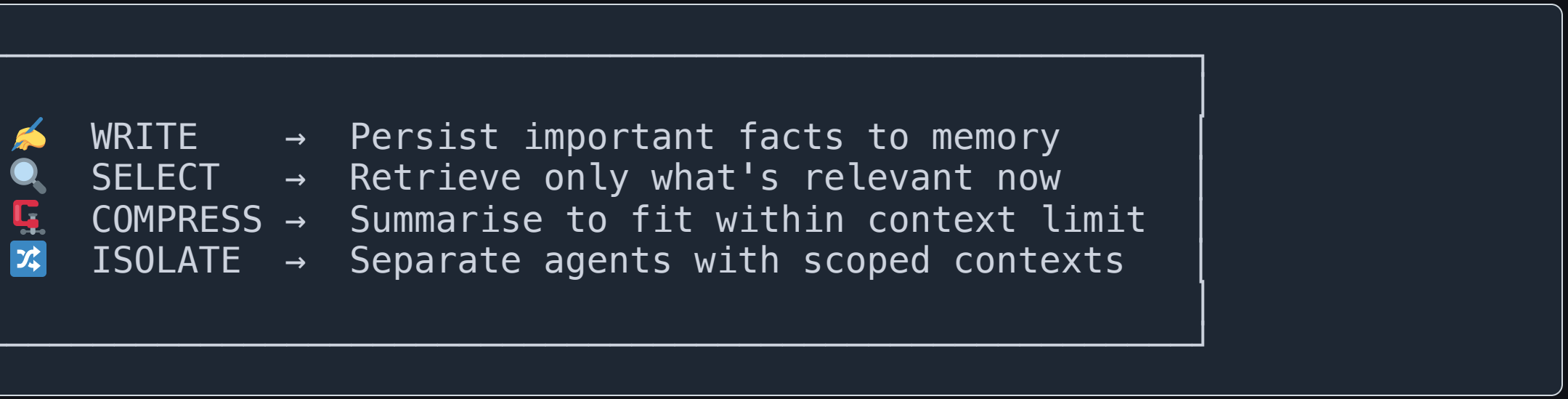
As conversations grow, agents face three critical failures:

Failure	What Happens	Impact
🧠 **No Memory**	Each turn starts fresh	User repeats account ID every time
💥 **Context Overflow**	Too many tokens in one prompt	API errors, high cost
🔀 **Context Pollution**	Irrelevant info fills the window	Poor, hallucinated answers

What is Contextual Engineering?

> *"The discipline of designing and managing the information presented to an LLM to maximise the quality of its outputs."*

Four core strategies:



This project implements **all four** in a banking customer experience agent.

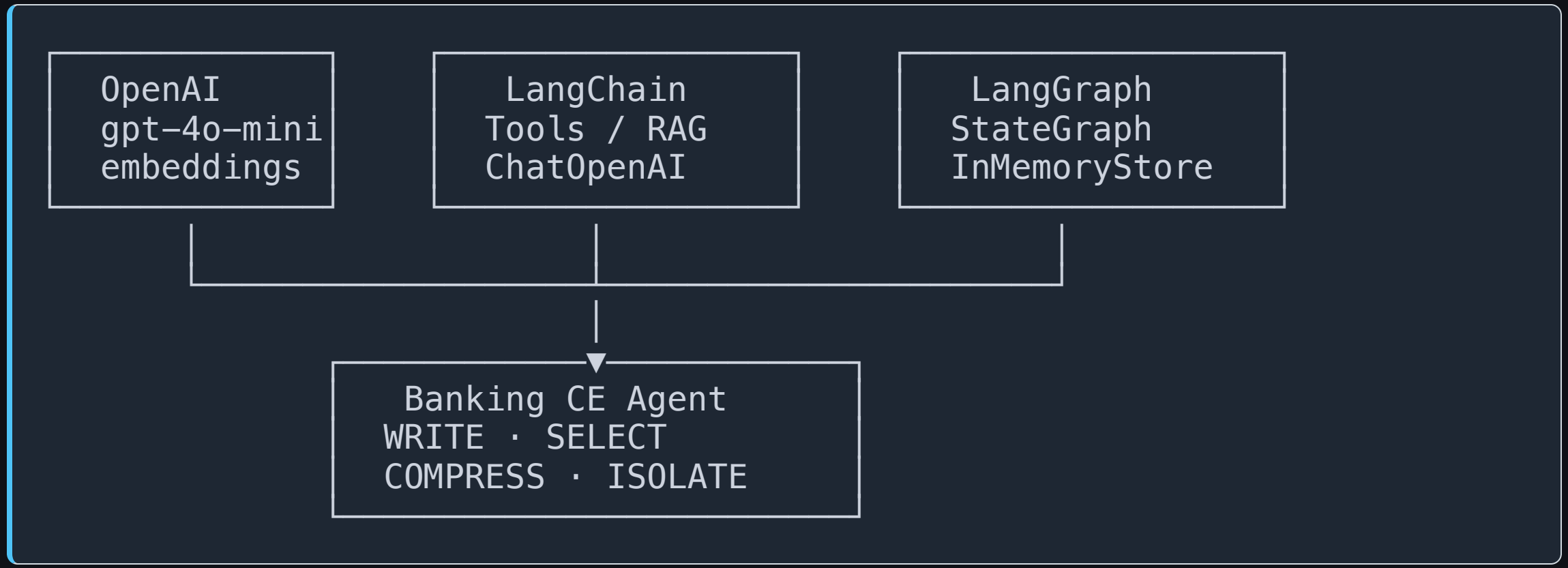
The Banking CE Agent

A fully functional banking assistant for **IndiaFirst Bank** that handles:

- 💰 Account balance & transaction history
- 🚨 Fraud detection & suspicious transaction flagging
- 🏠 Loan eligibility checks (personal & home loans)
- 📊 EMI calculations & Fixed Deposit rates
- 📱 UPI limit checks & transfer validation
- 🎟️ Support ticket creation
- 📖 Policy Q&A via RAG (regulatory, KYC, limits)

Three customers with realistic profiles: Rajesh Kumar, Priya Sharma, Amit Patel

Technology Stack



Library	Role
`langchain-openai`	LLM & embeddings

✍️ Strategy 1: WRITE

> *Persist important facts so the agent remembers them across turns and sessions.*

Two layers of memory:

Layer 1 — Session Scratchpad (StateGraph)

```
class BankingState(TypedDict):
    messages:           list    # full conversation
    current_account:    str     # ← active account, persisted across turns
    fraud_flags:        list    # ← flagged transactions this session
    loan_context:       dict    # ← loan evaluation in progress
    session_summary:    str     # rolling summary
    interaction_count: int      # turn counter
```

Layer 2 — Long-Term Cross-Session Store (InMemoryStore)

```
store.put(("banking_sessions", account_id), "context", {
```

🔍 Strategy 2: SELECT

> *Retrieve only the context that is relevant for the current query — don't dump everything in.*

Two SELECT mechanisms:

1. RAG — Policy Document Retrieval

```
vectorstore = FAISS.from_documents(policy_docs, OpenAIEmbeddings())
retriever   = vectorstore.as_retriever(search_kwargs={"k": 3})
```

The agent calls `search_banking_policy` only when it needs loan rules,
UPI limits, KYC requirements, or fraud detection thresholds.

2. Memory Injection — Prior Session Context

```
# Only injected if a prior summary exists for this account
```

Strategy 3: COMPRESS

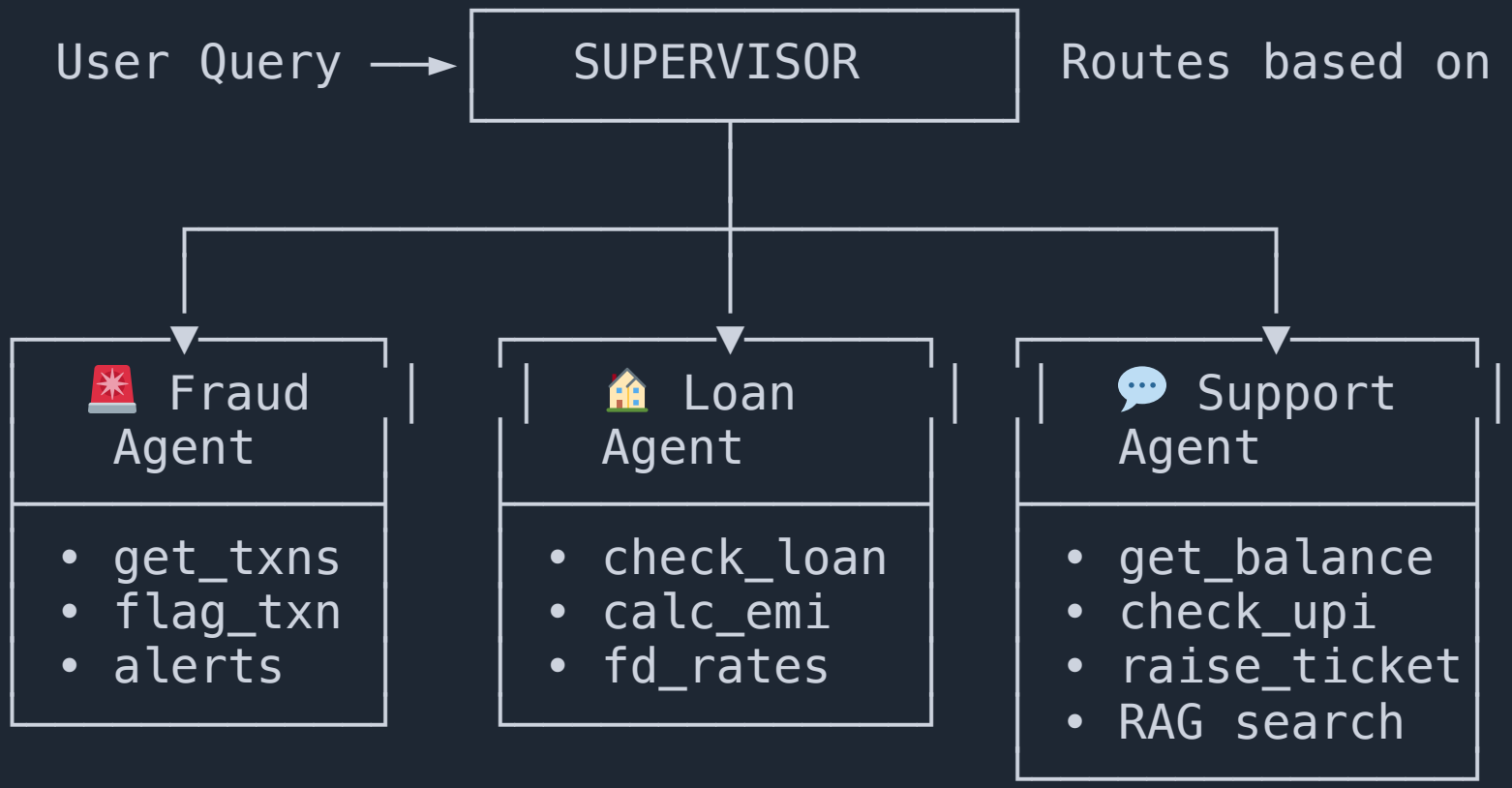
> *Summarise verbose history into a compact representation that fits within the context window.*

Triggered every 3 interactions per account:

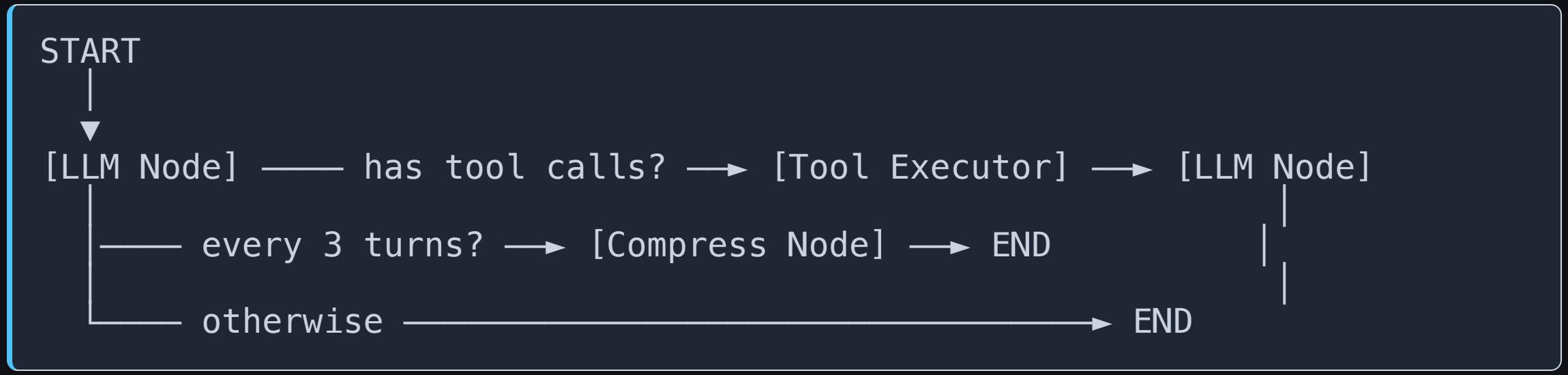
```
def compress_node(state, store):
    summary = llm.invoke([
        SystemMessage(content=COMPRESS_PROMPT),
        *state["messages"]
    ])

    # ✅ Save compact summary to long-term store
    store.put(("banking_sessions", account_id), "summary", summary)

    # ✅ Drop old messages — keep only last 6
    return {
        "session_summary": summary,
        "messages": state["messages"][-6:]
```

Strategy 4: ISOLATE

Give each specialist agent only the tools and context it needs — nothing more.



Single Agent — Graph Architecture

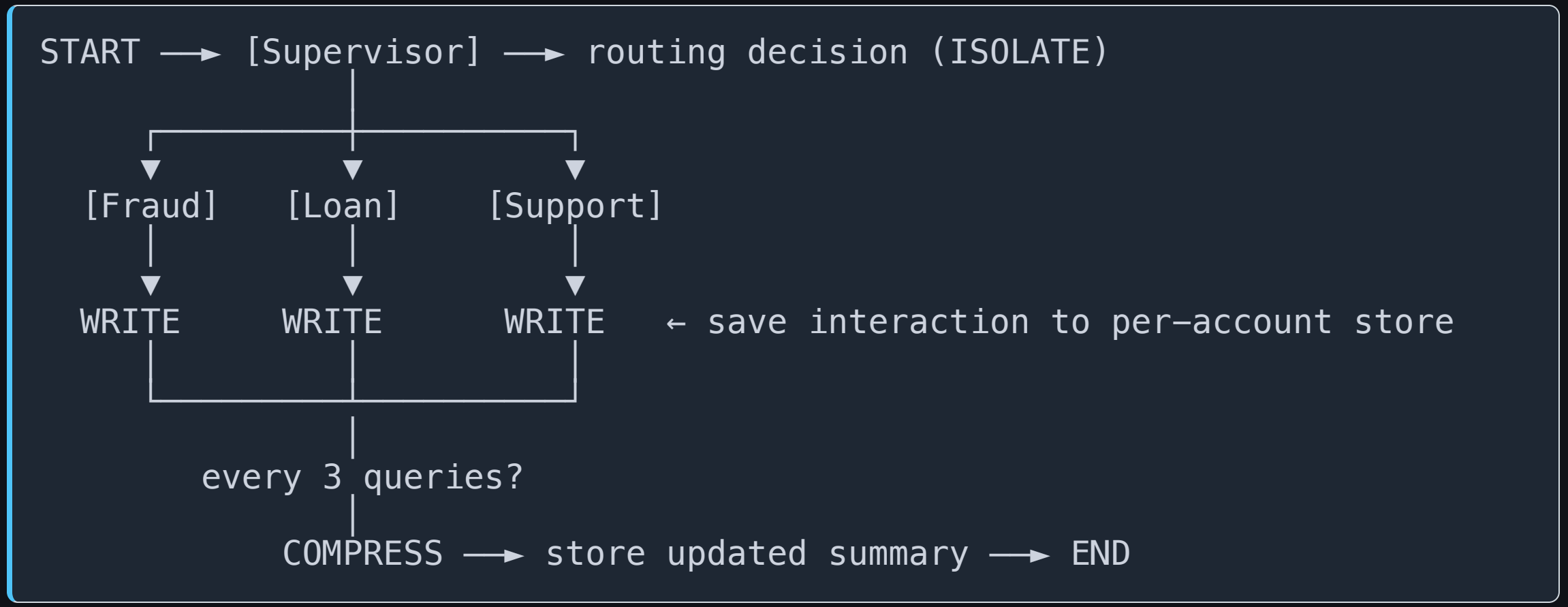


LLM Node does:

- Injects session scratchpad + prior memory (WRITE + SELECT)
- Calls LLM with all tools bound including RAG retriever (SELECT)
- Extracts and persists account ID from messages (WRITE)

Tool Executor does:

Supervisor — Multi-Agent Architecture



Account isolation: namespace `("supervisor_sessions", "ACC001")` is completely

🎬 Demo Flow — Single Agent

Turn	User	Concepts Active
1	"My account is ACC001. Check balance."	**WRITE** (stores ACC001)
2	"Suspicious Rs.1,20,000 transfer on Mar 12."	**SELECT** (loads ACC001 context), **WRITE** (flags fraud)
3	"Am I eligible for a Rs.10L personal loan?"	**COMPRESS** fires (turn 3), **SELECT** (RAG loan policy)
4	"EMI at 10.5% for 48 months?"	**WRITE** (loan context persisted)
5	"What are UPI limits for Rs.90,000 transfer?"	**SELECT** (RAG UPI policy)

Watch for in terminal:

🎬 Demo Flow — Supervisor Agent

Query	Account	Routed To	Concepts
"Rs.1,20,000 at 2AM — fraud?"	ACC001	🚨 Fraud Agent	**ISOLATE** routes, **WRITE** saves
"Eligible for home loan of Rs.50L?"	ACC001	🏠 Loan Agent	**ISOLATE** routes, **SELECT** injects ACC001 memory
"Balance + fraud alerts?"	ACC002	💬 Support Agent	**ISOLATE** routes, **WRITE** saves (separate namespace)

Key observation:

- ACC001 and ACC002 memories are **never mixed** — separate `InMemoryStore` namespaces
- Sub-agents receive **only HumanMessages** — supervisor routing calls are hidden

Code: WRITE Strategy

```python
# ── In banking_llm_node ──────────────────────────────────────
import re
current_account = state.get("current_account")

if not current_account:
    for msg in state["messages"]:
        match = re.search(r"ACC\d{3,}", str(msg.content))
        if match:
            current_account = match.group(0)  # ← auto-extract account ID
            break

if current_account:
    updates["current_account"] = current_account  # ← persist to scratchpad

# ── Cross-session long-term memory ──────────────────────────
store.put(
    ("banking_sessions", account_id),  # ← scoped by account_id
    "summary",
    {"summary": new_summary}
)
```

Code: SELECT & COMPRESS

```
# ── SELECT: RAG retrieval (only called when agent needs policy) ──
retriever_tool = Tool(
    name="search_banking_policy",
    func=lambda q: "\n\n".join(
        doc.page_content
        for doc in retriever.invoke(q)
    )
)

# ── SELECT: Memory injection ──────────────────────────────────────
memories = store.search(("banking_sessions", account_id))
if memories:
    system_prompt += f"\n[Prior session]\n{memories[0].value['summary']}"

# ── COMPRESS: Triggered every 3 turns ─────────────────────────────
if interaction_count % 3 == 0:
    summary = llm.invoke([SystemMessage(COMPRESS_PROMPT)] + messages)
    store.put(namespace, "summary", {"summary": summary.content})
    trimmed = messages[-6:]   # ← drop old messages after summarising
```

Code: ISOLATE Strategy

```
# Each agent gets ONLY its domain tools — no cross-contamination
fraud_agent = create_react_agent(
    model=llm,
    tools=[get_transaction_history,       # ← fraud domain only
           flag_suspicious_transaction,
           get_active_alerts],
    prompt=FRAUD_AGENT_PROMPT,
)

loan_agent = create_react_agent(
    model=llm,
    tools=[check_loan_eligibility,        # ← loan domain only
           calculate_emi, get_fd_rates],
)

# Sub-agents receive ONLY HumanMessages — supervisor routing is hidden
def _human_messages_with_context(state):
    return [m for m in state["messages"] if isinstance(m, HumanMessage)]
```

What Makes This Production-Quality

Feature	Implementation
Per-user memory isolation	Namespaced `InMemoryStore` keyed by `account_id`
No repeated account ID asks	Auto-extracted from messages via regex, stored in state
Valid tool call history	Every `tool_call_id` matched with a `ToolMessage`
RAG over policies	FAISS vector store, top-3 semantic retrieval
Context window management	Trim to last 6 messages after compression
Graceful fallback	Missing account ID → prompt only at tool call time
Multi-model support	Swap model string to use Claude, Gemini, etc.

Project Structure

```
agent-demo/
├── banking_ce_agent/
│   ├── banking_agent.py      ← Main agent: graph, nodes, strategies
│   ├── banking_tools.py      ← 10 banking tools (@tool decorated)
│   └── banking_knowledge.py  ← Policy docs for RAG (loans, fraud, UPI...)
├── requirements.txt           ← LangChain, LangGraph, FAISS, rich
├── .env                       ← OPENAI_API_KEY
└── readme.md
```

Run the demos:

```
# Single agent (WRITE · SELECT · COMPRESS)
python banking_ce_agent/banking_agent.py

# Supervisor multi-agent (ISOLATE · WRITE · SELECT · COMPRESS)
python banking_ce_agent/banking_agent.py supervisor

# Both demos sequentially
```

Key Takeaways

Contextual Engineering is not a framework — it's a discipline.

Without CE	With CE
Agent asks for account ID every turn	Account ID captured once, persisted for session
Loan policies hallucinated	Exact policy retrieved via RAG
Context grows → API errors	Rolling compression keeps window bounded
One agent knows too much	Each specialist agent isolated to its domain
User A sees User B's data	Per-user namespaced memory store

> **The quality of your agent's output is directly proportional to the quality of context you provide it.**

Thank You

Banking CE Agent — Full Implementation

GitHub: `github.com/RaviChandraSingam/agent-demo`

Four strategies, one demo:

```
✍️  WRITE    →  LangGraph StateGraph + InMemoryStore
🔍  SELECT   →  FAISS RAG + Memory injection
🗜️  COMPRESS →  Rolling summary + message trimming
🔀  ISOLATE  →  Supervisor + domain-scoped specialist agents
```

Run it yourself:

```
git clone https://github.com/RaviChandraSingam/agent-demo
cd agent-demo && python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
echo "OPENAI_API_KEY=sk-..." > .env
```